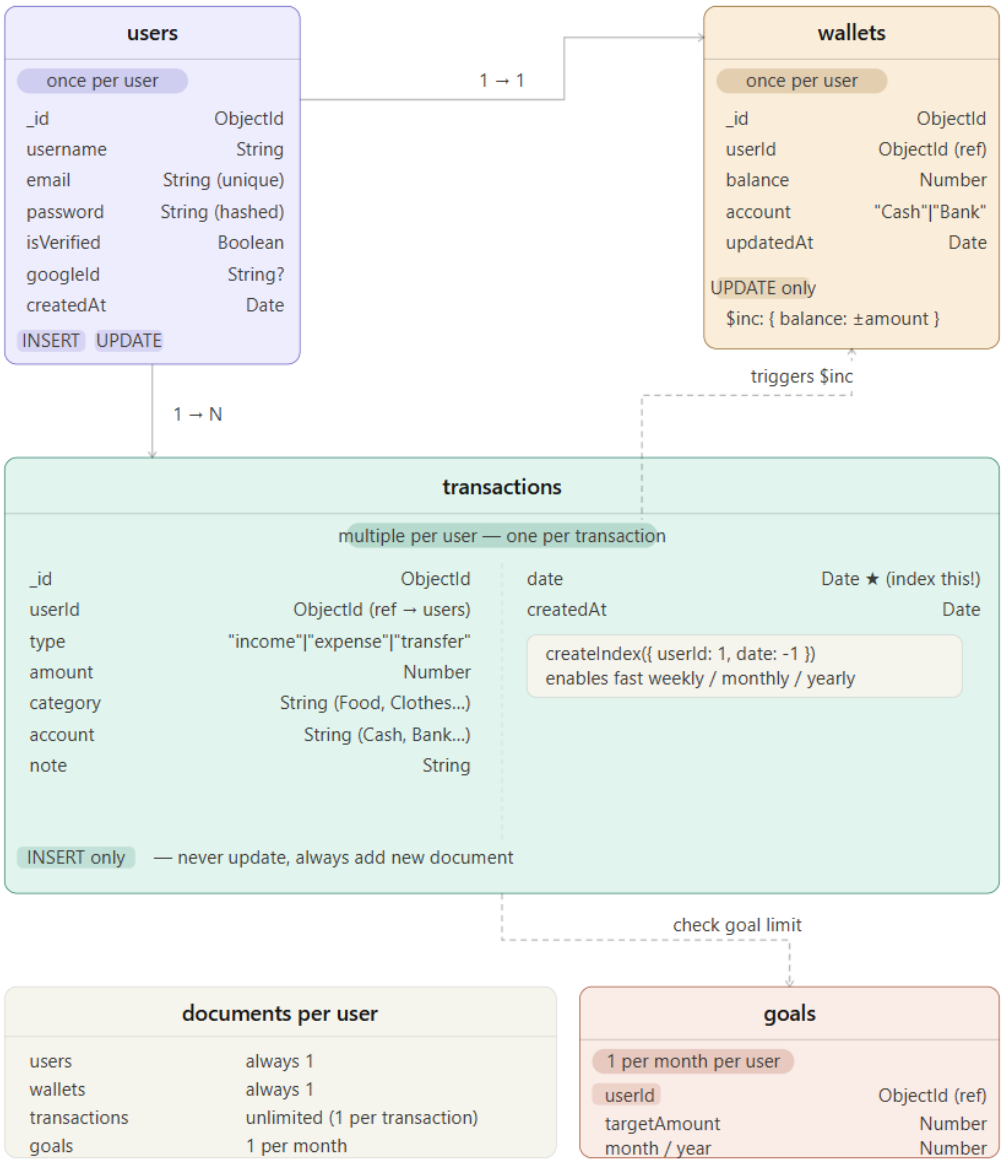
Expense Tracker — Backend Notes

Dococument Schema:



weekly and monthy data must be aggregated fom transaction document

---

1. MongoDB Collections

You need exactly 4 collections. Here is what each one stores and how many documents per user.

A. users

Purpose: Store signup / login info for every user.

Collection: users
Documents per user: Always 1 — one account per user
Operation: INSERT on signup | UPDATE on profile change

```
{ _id: ObjectId,
  username: String,
  email: String,       // must be unique
  password: String,    // bcrypt hashed
  isVerified: Boolean, // OTP verified?
  googleId: String,    // only if Google login
  createdAt: Date }
```

> *Password is hashed — never store plain text. Use bcrypt.*
> *Use JWT for authentication. Store token in cookie.*
> *For Google login use OAuth. googleId will be filled automatically.*

B. wallets

Purpose: Store the current balance of the user.

Collection: wallets
Documents per user: Always 1 — one wallet per user
Operation: UPDATE only — never INSERT a new one after first time

```
{ _id: ObjectId,
  userId: ObjectId,   // ref → users
  balance: Number,    // current balance
  account: String,    // "Cash" | "Bank"
  updatedAt: Date }
```

> *To subtract: db.wallets.updateOne({ userId }, { $inc: { balance: -500 } })*
> *To add: db.wallets.updateOne({ userId }, { $inc: { balance: +5000 } })*
> *$inc does the math automatically — no need to calculate manually.*

C. transactions

Purpose: Store every single expense and income entry.

Collection: transactions
Documents per user: Unlimited — every expense or income = 1 new document
Operation: INSERT only — never update old ones

```
{ _id: ObjectId,
  userId: ObjectId,    // ref → users
  type: String,        // "income" | "expense" | "transfer"
  amount: Number,
  category: String,    // "Food", "Clothes", "Bonus" etc.
  account: String,     // "Cash" | "Bank"
  note: String,
  date: Date,          // ★ KEY FIELD — index this!
  createdAt: Date }
```

> *★ The date field powers ALL weekly / monthly / yearly queries.*
> *Create index: db.transactions.createIndex({ userId: 1, date: -1 })*
> *Every time user adds expense → INSERT here + UPDATE wallets balance.*

D. goals

Purpose: Store how much the user wants to save each month.

Collection: goals
Documents per user: 1 per month — new document each month
Operation: INSERT for new month | UPDATE for same month

```
{ _id: ObjectId,
  userId: ObjectId,       // ref → users
  targetAmount: Number,   // savings goal
  month: Number,          // 1-12
  year: Number,           // e.g. 2026
  createdAt: Date }
```

> *Check if goal exists for this month — if yes UPDATE, if no INSERT.*

---

2. Documents Per User — Summary

users → always 1
wallets → always 1

transactions → unlimited (1 per expense/income)
goals → 1 per month

---

3. Date Queries — Weekly / Monthly / Yearly

All queries use the same aggregate pattern. Only the date range changes.

Weekly

```
const now = new Date();
const weekAgo = new Date();
weekAgo.setDate(now.getDate() - 7);
// filter: date: { $gte: weekAgo, $lte: now }
```

Monthly

```
const start = new Date(now.getFullYear(), now.getMonth(), 1);
// filter: date: { $gte: start, $lte: now }
```

Yearly

```
const start = new Date(now.getFullYear(), 0, 1);
// filter: date: { $gte: start, $lte: now }
```

Aggregate Pattern (same for all)

```
db.transactions.aggregate([
  { $match: { userId, type: "expense", date: { $gte, $lte } } },
  { $group: { _id: "$category", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

Useful MongoDB Date Operators

$dayOfWeek → get day (1=Sun … 7=Sat) — use for bar chart
$month → get month number (1–12) — use for yearly breakdown
$year → get year number
$week → get week number of year

> *You write the query ONCE. Just change startDate dynamically in backend.*

4. API Routes

Base URL: http://localhost:5000/api

Auth

```
POST   /api/auth/signup
POST   /api/auth/login
POST   /api/auth/verify-otp
```

Transactions

```
POST   /api/transactions              ← add new expense / income
GET    /api/transactions?filter=weekly
GET    /api/transactions?filter=monthly
GET    /api/transactions?filter=yearly
```

Wallet

```
GET    /api/wallet                    ← get current balance
```

Goals

```
POST   /api/goals                     ← set monthly goal
GET    /api/goals                     ← get current month goal
```

Report

```
GET    /api/report?filter=monthly     ← download report
GET    /api/report?filter=yearly
```

Profile

```
GET    /api/profile                   ← get user info
PUT    /api/profile                   ← update user info
```

5. Query Params

Query params go after ? in the URL.

```
GET /api/transactions?filter=weekly
GET /api/transactions?filter=monthly&type=expense
```

Access in Express

```
const { filter } = req.query;   // 'weekly' | 'monthly' | 'yearly'
const { type }   = req.query;   // 'expense' | 'income'
```

3 Ways to Send Data in Express

Query params → ?filter=weekly → req.query.filter → for filters
Route params → /expenses/:id → req.params.id → for specific item
Body → req.body → req.body.amount → for POST data

---

6. Flow on Every Expense

- User fills: date, amount, category, note
- INSERT new document in transactions collection
- UPDATE balance in wallets using $inc: { balance: -amount }
- Check goals collection — if expense exceeds goal show warning
- Send email notification to user

> *Both INSERT (transactions) and UPDATE (wallets) must happen together.*

---

7. Key Rules to Remember

- transactions → INSERT only. Never update old documents.
- wallets → UPDATE only. Never insert a new one per expense.
- Always index: { userId: 1, date: -1 } in transactions.
- Never store weekly/monthly totals — calculate with aggregate.
- date field in transactions powers all chart and report queries.
- $inc handles math automatically — no manual calculation needed.
- goals: same month = UPDATE, new month = INSERT.